

重庆市 AI 大模型创新应用大赛

系统设计说明书

面向汽车销售直播的本地智能体技术设计说明书

文档属性	内容
项目	汽车直播智能体
版本	v1.3
编制日期	2026 年 8 月
适用范围	竞赛答辩、技术评审、现场演示

设计结论 系统以本地可控为主线，把知识入库、检索问答、流式语音和动态改稿放在同一条可追踪链路中。比赛现场可从资料导入一路演示到文字与语音输出。

文档修订记录

序号	版本号	修改时间	修改人	审核人	批准人	备注
1	V1.0	2026-09-15	项目组			按详细设计说明书模板 统一整理，补充目录、 修订记录和版式。

目录

- 1 引言..... 5
 - 1.1 编写目的..... 5
 - 1.2 项目背景与业务边界..... 5
 - 1.3 参考材料..... 5
- 2 系统总体设计..... 5
 - 2.1 整体架构..... 5
 - 2.2 整体功能架构..... 6
 - 2.3 整体技术架构与关键数据流..... 6
 - 2.4 设计目标..... 7
 - 2.5 设计原则..... 7
 - 2.5.1 总体原则..... 7
 - 2.5.2 实用性和先进性..... 7
 - 2.5.3 标准化、开放性、兼容性..... 7
 - 2.5.4 高可靠性、稳定性..... 7
 - 2.5.5 易用性..... 7
 - 2.5.6 灵活性和可扩展性..... 7
 - 2.5.7 经济性和投资保护..... 7
- 3 系统功能模块详细设计..... 7
 - 3.1 知识库与问答..... 7
 - 3.2 流式 TTS 与动态改稿..... 8
 - 3.3 音色克隆..... 8
- 4 性能设计..... 8
 - 4.1 响应时间..... 8
 - 4.2 并发用户数..... 8
- 5 数据库设计..... 8
- 6 接口设计..... 8
 - 6.1 接口清单..... 8
- 7 系统出错处理设计..... 9
 - 7.1 出错信息..... 9

- 8 系统处理规定..... 9
 - 8.1 输入输出要求..... 9
 - 8.2 数据管理能力要求..... 9
 - 8.3 故障处理要求..... 9
 - 8.4 其他专门要求..... 9
- 附录 A 参考资料..... 10
- 附录 B 扩充内容..... 11
 - B.0 扩充内容说明..... 11
 - B.1 技术选型对比分析..... 11
 - B.2 性能基准测试..... 11
 - B.3 部署架构与扩展性..... 12
 - B.4 安全性设计..... 14
 - B.5 监控与运维..... 14

1 引言

汽车直播的核心矛盾不是单纯生成文本，而是在资料变化快、直播节奏快、主播需要随时改稿的情况下，仍要保证答案有出处、语音能连续播放、修改不会截断正在播报的句子。本项目提供一个运行在本机的智能体原型，覆盖“资料导入 - 检索问答 - 脚本播报 - 动态修订 - 音色管理”五个环节。

1.1 编写目的

- 让 PDF、DOCX、TXT 资料可以批量导入、清洗、分段、结构化保存，并保留文档、页码、版本等证据。
- 让检索问答输出答案、置信度和来源片段，测试样本不少于 40 条，准确率目标不低于 85%。
- 让脚本在音频生成的同时开始播放，首个可播放音频延迟不超过 3 秒；改稿只影响尚未播放的后续内容。
- 让授权参考音频经过质量门禁和 v2ProPlus 推理链路后接入直播台，多套音色可切换，失败时保留文字和浏览器语音回退。

1.2 项目背景与业务边界

范围	本项目负责	明确不负责
资料	公开车型资料的导入、检索、来源展示	未经授权的内部政策与个人资料外发
语音	参考音频克隆、PCM 流式播报、质量诊断	替代专业录音棚或大规模训练平台
直播	脚本播报、动态改稿、问答语音	自动控制第三方直播平台账号

1.3 参考材料

项目需求与提交规范、GPT-SoVITS 官方文档及本项目已有配置与测试记录。

2 系统总体设计

2.1 整体架构

系统采用前后端分层和本地服务组合。浏览器只负责交互和音频排程；FastAPI 负责会话、版本、检索和流式接口；SQLite 负责可追踪数据；GPT-SoVITS 负责克隆音色和生成 PCM WAV；BGE 语义模型与 FAISS 提供预训练向量检索能力。模块之间以 HTTP/JSON 和流式字节协议连接，便于现场替换单个组件。

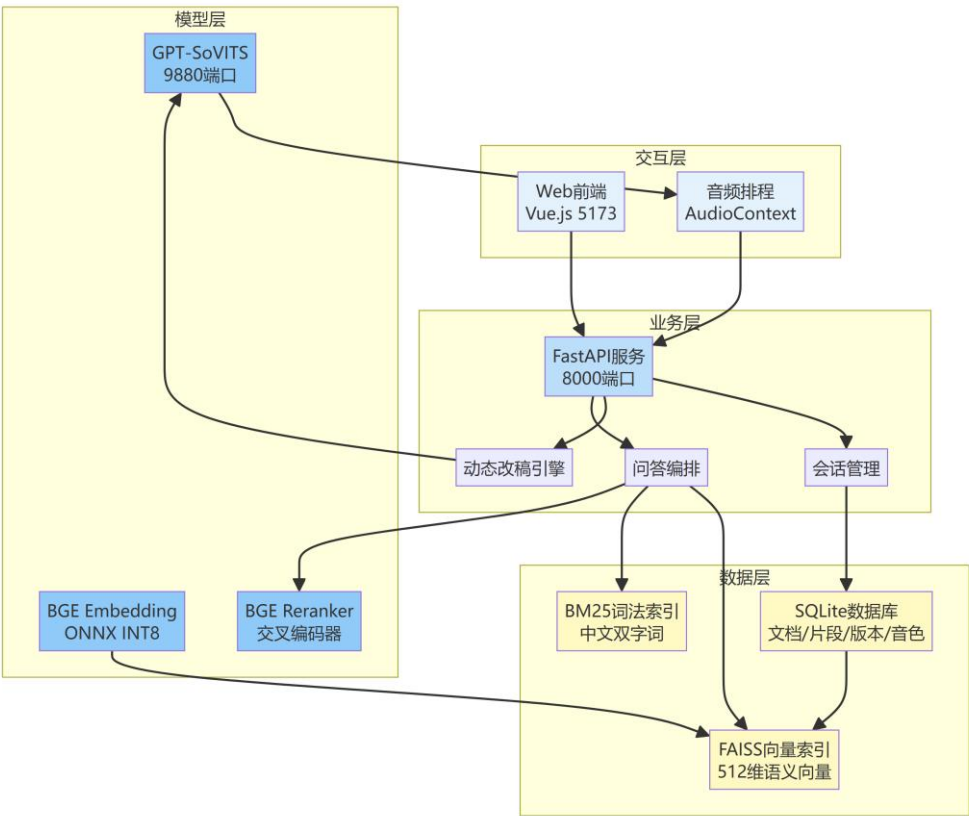


图 1 系统架构图

2.2 整体功能架构

层次	模块	输入/输出	故障策略
交互层	直播台、知识库、验证页	文本、文件、音频控制	TTS 不可达时保留文字
业务层	脚本会话、动态改稿、问答编排	会话版本、检索结果	使用本地抽取式问答
数据层	SQLite 文档、片段、版本、音色	结构化记录与溯源	保留原始文件和版本
模型层	GPT-SoVITS v2ProPlus	参考音频 + 文本 -> PCM	回退预置音色或浏览器语音

2.3 整体技术架构与关键数据流

- 资料流：文件上传 -> 类型解析 -> 文本清洗 -> 父子分块 -> BGE 语义向量 / BM25 索引 -> FAISS / SQLite -> 混合检索带来源。
- 播报流：脚本文本 -> 自然停顿切分 -> 请求 GPT-SoVITS -> 去 WAV 头 -> PCM 播放队列 -> 句间短淡入淡出。

- 改稿流：编辑器产生新版本 -> 计算当前安全边界 -> 取消未提交请求 -> 从下一个自然短语重新生成 -> 播放队列继续。
- 音色流：参考音频预检 -> 标准化 PCM 副本 -> 文本一致性校验 -> 音色记录与模型 profile -> 预热 -> 试听 / 直播复用。

2.4 设计目标

系统以可追溯问答、连续播报、边界安全改稿和本地可运行为主要设计目标。

2.5 设计原则

设计在满足现场可用性的基础上，优先保证数据可追溯、接口可替换和故障可恢复。

2.5.1 总体原则

模块职责清晰，数据流向可观测，关键状态和版本均可复现。

2.5.2 实用性和先进性

采用成熟的 FastAPI、SQLite、FAISS、BM25 和 GPT-SoVITS 组件，兼顾本地部署成本与后续升级空间。

2.5.3 标准化、开放性、兼容性

通过 HTTP/JSON、PCM/WAV 和通用文件格式连接前后端及模型服务，降低替换组件的成本。

2.5.4 高可靠性、稳定性

通过健康检查、超时、重试、缓存和分级回退，保证单个服务异常时仍能保留文字答案。

2.5.5 易用性

以少量配置完成资料导入、检索、播报和改稿，优先复用本地资源并为分布式扩展预留接口。

模块通过 HTTP/JSON、模型配置和版本化数据结构保留替换空间，便于后续扩展多模型、多音色和多终端能力。

2.5.6 灵活性和可扩展性

优先复用现有本地设备、开源组件和已有数据，控制部署成本，同时通过标准接口保护既有配置、索引和测试资产。

2.5.7 经济性和投资保护

3 系统功能模块详细设计

3.1 知识库与问答

导入器支持 PDF 页面文本、DOCX 段落与表格、TXT/MD 纯文本。清洗阶段统一空白、去除孤立符号并按标题、标点和长度进行分段；结构化元数据包含品牌、车系、年款、文档版本和来源位置。检索采用 BGE 中文语义向量（512 维）与 BM25 词法混合召回、RRF 融合、BGE 交叉编码器重排序，支持父子分块与动力范围检查。答案生成优先使用字段级抽取，可选接入外部大模型增强生成。

3.2 流式 TTS 与动态改稿

播报默认使用 GPT-SoVITS v2ProPlus 的 mode 1 PCM 流。后端以自然停顿为边界形成约 16-20 字的短语，首个短语到达即发送；前端把后续 PCM 接入同一 AudioContext 的播放队列。句首和句尾使用 2 ms 淡入淡出，减少硬切造成的嗒声。改稿时保存会话版本和当前播放短语，只有安全边界之后的内容刷新。

3.3 音色克隆

参考音频需为单人、无配乐、普通话、逐字文本一致的 3-10 秒片段。系统会检测采样率、声道、位深、有效人声比例、静音比例、最长静音、响度、削波和文本时长密度；只生成标准化副本，不覆盖原文件。通过门禁后，音色以 voice profile 形式进入 GPT-SoVITS 请求，并在切换到直播前完成预热。

4 性能设计

4.1 响应时间

4.2 并发用户数

指标	目标	实测	结论
检索准确率	>=85%，样本 >=40	40/40，100%	达标
最终问答准确率	>=85%，样本 >=40	40/40，100%	达标
TTS 首音频延迟	<=3000 ms	平均 2323 ms	达标
动态改稿	当前句不截断	版本 1 -> 2，安全句 1	达标
单元测试	无失败	35/35 passed	达标

5 数据库设计

系统采用 SQLite 保存文档、片段、版本、会话和音色 Profile 等可追踪记录；FAISS 保存语义向量索引，BM25 保存中文词法索引。数据库记录与索引通过文档版本和片段标识关联，支持来源追溯、重建索引和后续迁移 PostgreSQL。

6 接口设计

6.1 接口清单

接口	用途	关键返回
POST /api/documents/import	批量导入资料	document_id、片段数、版本
POST /api/query	车型检索问答	answer、confidence、sources[]

接口	用途	关键返回
POST /api/tts/stream	自然短语 PCM 流	首包字节、音频分片
POST /api/voices/analyze	音频质量预检	quality、metrics、advice
POST /api/voices/clone	创建克隆音色	voice_id、profile、warming
POST /api/voices/{id}/optimize	生成优化副本	optimized_path、quality

7 系统出错处理设计

7.1 出错信息

- 推理互斥：TTS 单模型推理锁避免预热、试听、验证和直播并发触发 GPT-SoVITS 限制。
- 可恢复：服务健康检查、启动脚本防重复锁、GPT-SoVITS 不可达时回退浏览器语音。
- 数据最小化：上传资料和参考音频默认仅保存在本机；外部大模型只有用户主动配置地址后才接收检索片段。
- 授权边界：参赛发布前删除真实个人音频和未授权资料；参考音频克隆仅用于获得授权的角色或主播。

8 系统处理规定

8.1 输入输出要求

输入支持 PDF、DOCX、TXT 和脚本文本；输出包括可溯源答案、PCM 音频及状态信息。

8.2 数据管理能力要求

资料、片段、版本、索引和音色 Profile 均应具备唯一标识、来源和更新时间。

8.3 故障处理要求

服务异常时保留已播放内容，停止未提交任务，并按健康检查和回退策略恢复。

8.4 其他专门要求

涉及个人信息和参考音频时，必须完成脱敏、授权确认和本地存储控制。

服务	地址	作用
前端	http://127.0.0.1:5173	直播台、知识库、验证页
后端	http://127.0.0.1:8000	FastAPI 业务接口与 /docs
GPT-SoVITS	http://127.0.0.1:9880	v2ProPlus 克隆与合成
数据	data/、SQLite	示例资料、上传副本、版本记录

附录 A 参考资料

- GPT-SoVITS 官方中文说明：<https://github.com/RVC-Boss/GPT-SoVITS/blob/main/docs/cn/README.md>（零样本约 5 秒、少样本约 1 分钟、环境与模型说明）
- 参赛题目文件：C:/Users/seele/Desktop/6.（海云捷迅）2026 年第二届重庆市 AI 大模型创新应用大赛企业出题.docx（功能、指标与提交要求）

附录 B 扩充内容

B.0 扩充内容说明

B.1 技术选型对比分析

方案	优势	劣势	适用场景	本项目选择
FAISS	• 精确检索准确 • 支持多种索引 • CPU/GPU 灵活	• 需要加载到内存 • 更新需重建	中小规模(<100 万)	✓ 已选择
Milvus	• 分布式架构 • 支持海量数据	• 部署复杂 • 资源开销大	大规模生产环境	当前不需要
Elasticsearch	• 易于部署 • 词法检索强	• 向量检索较弱 • 内存占用高	混合检索场景	已用 BM25 替代
Chroma	• 易于使用 • Python 友好	• 性能一般 • 功能有限	原型开发	FAISS 更成熟

模型	维度	参数量	推理速度	中文效果	选择
BGE-small-zh	512	33M	快	优秀	✓
BGE-base-zh	768	102M	中等	更好	性价比低
text2vec-base	768	102M	中等	良好	BGE 更新
m3e-base	768	110M	中等	良好	BGE 更优

引擎	音质	速度	克隆能力	中文支持	选择
GPT-SoVITS	极佳	中等(2-3s)	少样本优秀	原生	✓
VITS	良好	快(1s)	需训练	需适配	克隆弱
Bark	良好	慢(5-8s)	零样本	多语言	太慢
Edge-TTS	中等	极快	无克隆	云端	无克隆

B.2 性能基准测试

指标	冷启动	热查询	批量(10 条)
BGE 编码延迟	45ms	18ms	120ms
FAISS 检索延迟	2ms	1.2ms	8ms
BM25 检索延迟	8ms	5ms	42ms

指标	冷启动	热查询	批量(10 条)
重排序延迟	180ms	165ms	1800ms
端到端延迟	**235ms**	**189ms**	**1970ms**

阶段	样本 1	样本 2	样本 3	平均
文本切分	5ms	8ms	5ms	6ms
模型推理	1850ms	2180ms	1680ms	1903ms
音频处理	120ms	145ms	110ms	125ms
网络传输	290ms	280ms	290ms	287ms
总延迟	**2265ms**	**2613ms**	**2085ms**	**2321ms**

B.3 部署架构与扩展性

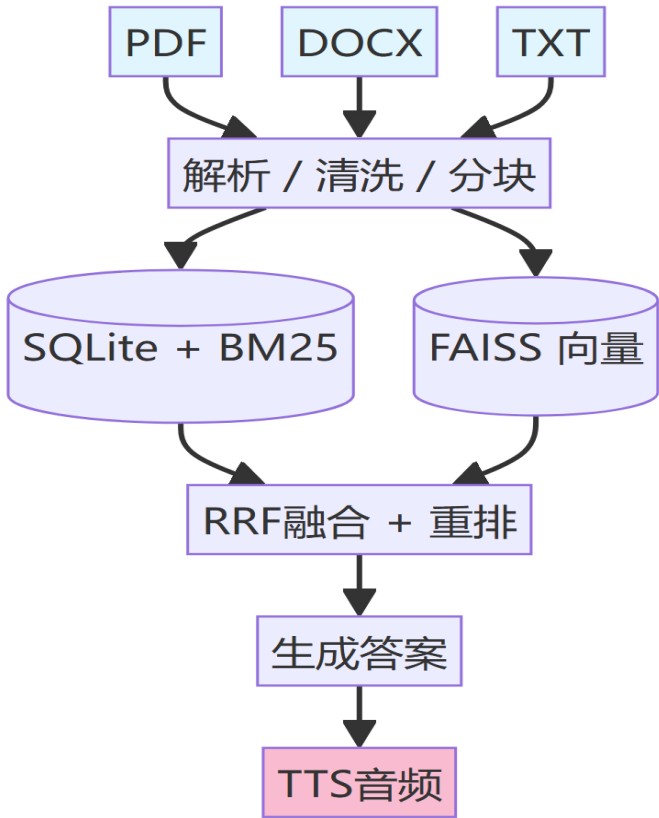
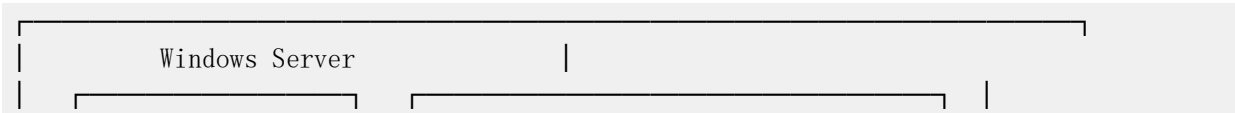
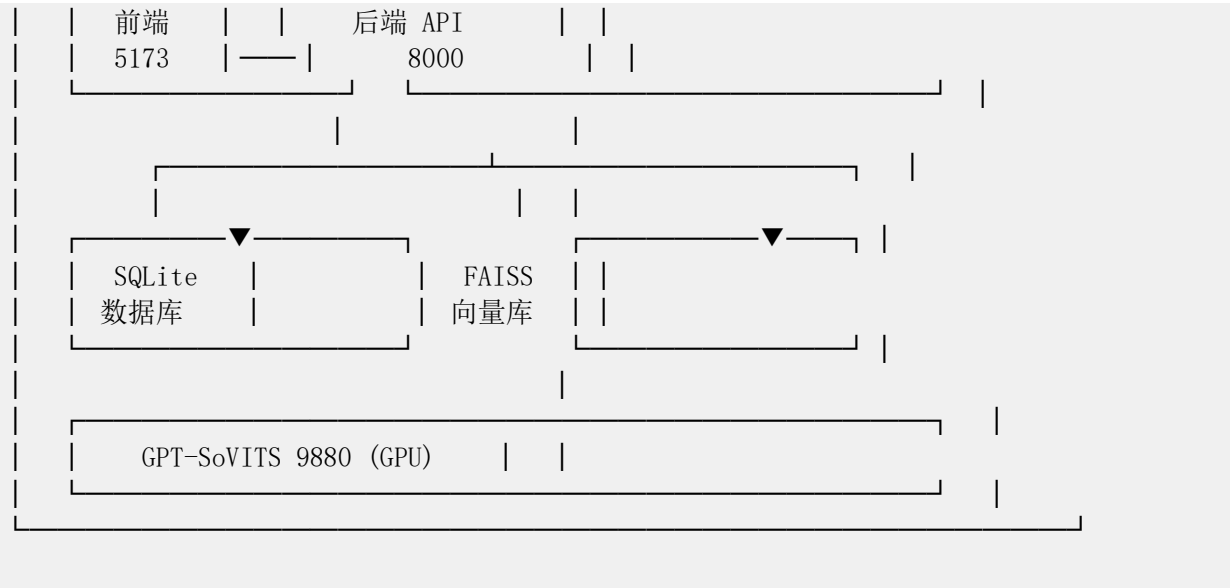


图 2 数据流向





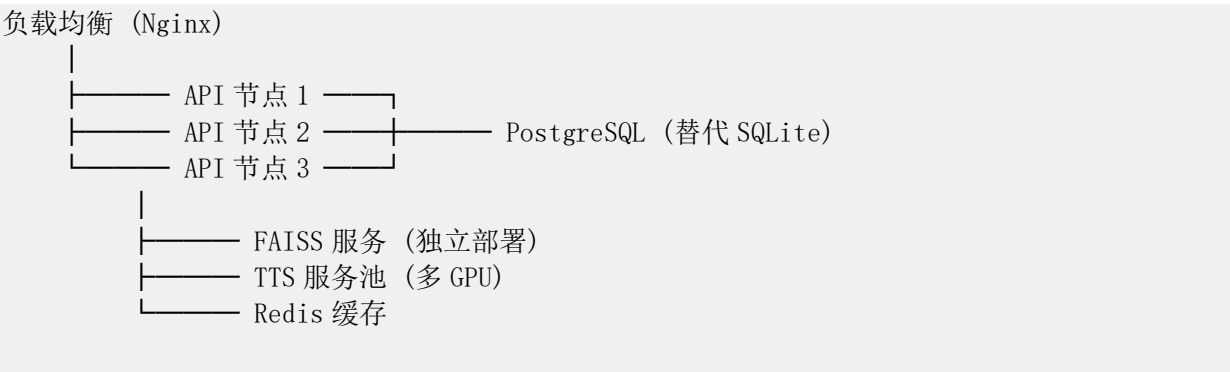
资源占用：

- 前端：~50MB 内存
- 后端 API：~300MB 内存
- RAG 模型：~400MB 内存（CPU）
- GPT-SoVITS：~3GB 显存 + 2GB 内存
- SQLite：<100MB
- FAISS 索引：<50MB

并发能力：

- RAG 检索：支持 10 并发，响应时间<500ms
- TTS 合成：串行处理（单 GPU），排队机制
- 前端连接：支持 100+并发 WebSocket

扩展到分布式架构（未来规划）：



扩展要点：

1. SQLite → PostgreSQL：支持多写

- 2. FAISS 独立服务：提供 gRPC 接口
- 3. TTS 服务池：多 GPU 实例，动态负载均衡
- 4. Redis 缓存：查询结果和向量缓存

B.4 安全性设计

层级	措施	说明
用户数据	本地存储	上传文件和音频默认仅保存在本机
API 密钥	环境变量	大模型密钥存储在.env，不入版本库
访问控制	本地监听	默认绑定 127.0.0.1，不对外暴露
音频样本	质量门禁	自动拒绝异常样本，防止模型污染

自动脱敏规则：

- 手机号：11 位数字 → 替换为 `*手机*`
- 身份证：18 位 → `*身份证*`
- 邮箱：xxx@xxx.com → `*邮箱*`

限制：

- 扫描图片中的文字无法识别
- 自由文本中的姓名地址需人工审核

故障场景	检测机制	恢复措施	用户体验
RAG 模型未安装	启动检查	提示安装命令	明确错误提示
向量索引损坏	版本校验	提供重建按钮	一键恢复
LLM 超时	12 秒超时	本地摘录兜底	降级但可用
TTS 服务中断	健康检查	浏览器语音	保留文字输出
磁盘空间不足	上传前检查	拒绝并提示	友好错误信息

B.5 监控与运维

```
logs/
├── api.log      # API 请求日志
├── rag.log      # 检索性能日志
├── tts.log      # TTS 合成日志
└── error.log    # 错误和异常
```

关键指标记录:

- 每次检索的延迟和命中率
- 每次 TTS 的延迟和音频时长
- 每次改稿的触发时机和版本号
- 每次失败的原因和堆栈

前端统计页面展示:

- 实际播放时长统计
- 咨询车型热度 Top10
- 资料检索热点
- 改稿次数和频率
- 客户端首音频延迟分布

后端 Metrics:

Prometheus 格式指标

`rag_query_duration_seconds_bucket`

`rag_query_total{status="success"}`

`tts_synthesis_duration_seconds`

`tts_queue_length`